

AI Data Analysis using Full Stack Development

1st Kondeti Dillip Kumar

*Assistant Professor, dept. Computer
Science and Engineering,
Dayanand Sagar Academy of
technology and management
Bangalore, India
email*

2nd Manjunath Maruti Naik

*Student, dept. Computer Science
and Engineering,
Dayanand Sagar Academy of
technology and management
Bangalore, India
email*

3rd Mallikarjun

*Student, dept. Computer Science and
Engineering,
Dayanand Sagar Academy of
technology and management
Bangalore, India
email*

4th Manjunath Gowda Y

*Student, dept. Computer Science and
Engineering,
Dayanand Sagar Academy of technology
and management
Bangalore, India
email*

5th Maddi Meghanath

*Student, dept. Computer Science and
Engineering,
Dayanand Sagar Academy of technology
and management
Bangalore, India
email*

Abstract— The exponential increase in Data Driven Decision Making (DDDM) has led to a tremendous increase in the demand for accessible, intelligent analysis platforms. This paper describes a full stack Django web application called the AI Data Analysis Platform (AIDAP), which integrates a variety of automated data preprocessing and analytic techniques (descriptive statistics, interactive visualization, automated machine learning [AutoML], and generative AI powered by the Google Gemini API) in order to provide the ability for any user without programming expertise to upload their CSV datasets and perform a complete analysis by utilizing a professional dashboard interface. The system provides support for automated task type detection (classification and regression), Scikit-learn model training, model comparisons, and the generation of predictions from TD model results. The NLP (natural language processing) Assistant allows users to interactively query their data, charts, and model results conversationally. When the AIDAP is evaluated against five benchmark datasets, it demonstrates extremely strong AutoML performance, with the best-performing model achieving 92.4% accuracy with respect to classification, 78% reduction of time spent to prepare for analysis and an average user satisfaction rating of 4.3 out of 5.0. The AIDAP demonstrates how to create a practical, production-grade analytical tool through the integration of Django, Pandas, Scikit-learn, Plotly, and Gemini AI.

Keywords: Django; AutoML; machine learning; data visualization; Gemini AI; NLP Assistant; Plotly; Scikit-learn; full-stack web application; data preprocessing.

I. INTRODUCTION

The significance of analysis of data is growing more and more among different sectors, including healthcare, education, business intelligence, scientific research, among others, as seen from the number of data-focused positions within these sectors. The process

of analysis of data can be technically challenging for a non-technical individual since traditional methods like software tools used in statistics, machine learning frameworks, and programming environments [1] are used.

AI Data Analysis Platform comes in to provide a solution to this challenge by providing a structured and well-arranged web application dashboard that hides away the technicalities of the analytical workflow. Using this platform, the user can upload a CSV file to obtain some descriptive statistics; plot graphs using Plotly [3]; automatically generate machine learning models using Scikit-Learn by automating the training process [5]; use inference to predict future values and interact with Google Gemini [4], a conversational AI assistant.

Moreover, all functionalities supported by this platform to execute each task within the analysis life cycle (e.g., preprocessing, visualization of the data, best model selection, and insight generation) are accessible through the sidebar-based UI without writing a single line of code.

The main contributions of this work are as follows: (1) a modular Django architecture incorporating data analytics, ML, and generative AI [2]; (2) an automatic preprocessing pipeline capable of managing missing or invalid records, outliers, and encoding [5]; (3) the Gemini assistance AI system that is able to access and extract live information about the dataset being analyzed [4]; (4) responsive SaaS UI dashboard; and (5) practical evaluation on real-world datasets [6].

II. LITERATURE SURVAY

Excel and SPSS have always been used for performing traditional data analysis, but both of them require an expert-level user. The advent of Python programming libraries such as Pandas and NumPy has made programming more convenient for data manipulation. However, they remain complicated for many users who do not know how to code.

Modern business intelligence applications like Tableau and PowerBI have made data visualization more convenient, yet they lack advanced machine learning (ML) features as well as support for conversational AI. Some of the existing automated ML systems (e.g., Auto-Sklearn, TPOT) simplify the task of choosing ML models by automating their creation; however, they operate separately without providing any convenient web application interface.

In light of the emergence of recently developed technologies such as Google's Gemini and OpenAI GPT-4, where there are data source natural language interfaces available, a number of academics have decided to build applications that incorporate LLMs into their data analysis toolset. However, only very few cutting-edge studies managed to come up with an open-source solution that combines all of these three features in one coherent solution. This is where the present project comes in, as it seeks to integrate these four aspects together in one Django application.

III. SYSTEM ARCHITECTURE

A. Overview

The platform is based on the Django’s MVT architecture, which forms the platform as a single application (called dataapp) through which all processing of analytical and Machine Learning tasks can take place at the backend in Python, while HTML templates of Django framework render the output of such tasks in Plotly.js library along with custom CSS/JS [3]. All communication amongst Components of the platform would take place via Django URLs and session management techniques of the same.

Component	Technology
Backend Framework	Django (Python)
Frontend	HTML5, CSS3, JavaScript
Data Processing	Pandas [2], NumPy
Machine Learning	Scikit-learn [5], Joblib
Visualization	Plotly [3], Matplotlib, Seaborn
AI Integration	Google Gemini API [4]
Database	Django ORM (SQLite/PostgreSQL)

B.Module Architecture:

The Codebase is made up of distinct modules according to the analytics pipeline:

- views.py (renders pages, uploads files, connects to AI chat endpoint, trains machine learning model, and predicts using it)
- services/ai_assistant.py (constructs requests to the Gemini API and builds the model that will be fed these requests)
- preprocessing.py (prepares and processes data)
- visualization/charts.py (renders charts using Plotly)
- models.py (stores details on all datasets, models, and predictions).

C. Upload and Manage Dataset/Session:

The user will be able to upload the CSV file via the web interface. After uploading the CSV file via the web interface, the system will use the Pandas library for reading the content of the CSV file, inferring the data type of each column, counting missing data values, and storing structural metadata of columns in the database. In the end, the DataFrame generated by the above operations will be stored in the Django Session. Thus, no other downstream module such as statistics, visualizations, and machine learning will have to perform any other operation on the DataFrame. [1]

D. Pre-processing Pipeline:

The preprocessing module contains a number of cleaning steps. These cleaning steps follow data engineering principles such that: [5]

- Imputing missing data points by mean, median, mode [5]
- Detection and removal of duplicates
- Outlier detection through IQR and Z-score [6]
- Label encoding for categorical features [5]
- Feature scaling using MinMaxScaler and StandardScaler [5]
- Converting data types and normalization of the dataset.

E. Statistical Module on Dataset:

Statistical information was calculated for all number-only columns in this data set using Pandas [2] and NumPy. Metrics included in this report were: mean, median, mode, variance, standard deviation, min, max, quartiles (Q1 and Q3), skewness and kurtosis. The metrics are shown in a style-enabled dashboard style table allowing users to assess quickly the distribution of data and identify outliers.

F. Visualization module:

The Visualization module uses Plotly to generate visually interactive charts using JSON objects on the server-side and then use Plotly.js to declare all those charts interactively through the web. List of chart types includes: Bar charts, Line charts, Scatter plots, Heat maps, Histogram, Box plot. Charts will also provide zoom, pan, filter, and export to PNG or SVG options. All charts will use responsive containers to render correctly according to display (desktop, tablet, and mobile).

G. Correlation module:

The Correlation module computes Pearson correlation coefficients for all numeric columns of the dataset. Through use of Pandas, the user will get the same result as that from the correlation module (pearson correlation matrix) only that the result from pandas will be plotted interactively in the form of a heatmap using Seaborn and then plotted on the web through Plotly.

H. AI Understand module:

Understand AI Module will use the Gemini API with an informal prompt together with metadata of the data set. This is including; names of columns, column types, sample rows, number of missing values, and domain that was inferred. This output will include an analysis of the type of the data set, machine learning path, important features, data quality, whether there is class imbalance, and the suggested models. This module of AI Understand will make the platform accessible to all users who have no idea about the concept of machine learning.

I. AutoML module:

"The automated machine learning engine" attempts to determine whether the problem at hand is either a classification problem or a regression problem from the cardinality and the data type of the columns used.

If the problem is a classification problem, then the engine will try using Random Forest, Support Vector Machines (SVM), Logistic Regression, K-Nearest Neighbor (KNN), and Decision Tree in its attempt to classify the data set. However, in the case where the problem at hand is that of regression, then the engine will use Linear, Ridge, and Lasso Regressions, as well as Gradient Boosting Regression.

J. Prediction module:

The predictive tool makes use of this serialized object to obtain the most optimized model, and the user's data is also incorporated (manually entered or imported from CSV files).

The user's data is processed using the same data transformations, such as scaled values and category encoding, as was done during training of the models; therefore, the way data is prepared for prediction is similar to the way it is prepared during training. The outputs from the prediction tool will be presented as tables on the application dashboard.

K. Gemini AI Assistant:

The conversational AI assistant is based on the Google Gemini API [4]. It is provided with a dynamically generated context object that includes: dataset size, columns and data type, target column, best model's name and accuracy, number of missing values, most correlated columns, numerical columns list, and categorical columns list. Thanks to such a contextual prompting technique, Gemini can generate dataset-specific outputs instead of general ones.

The connector deals with several possible errors, including: missing API key, wrong credentials, quota reached, unavailable model, response time exceeded, empty response, failed connection, and missing Gemini package [4]. All errors will be displayed as friendly messages for users on the dashboard

IV. Methodology

The methodology used in the project involves various steps.

Step 1: Data Set Upload:

Datasets are uploaded in CSV or Excel format by users.

Step 2: Data Preprocessing:

The process involves cleaning up the data through:

Dealing with missing data

Eliminating duplicates

Anomaly detection

Standardization of numeric variables

Step 3: Exploratory Data Analysis:

Calculations for the following are performed:

Mean

Median

Variance

Correlation

Data distribution

Step 4: Visualization:

Graphs and charts are automatically plotted.

Step 5: Insight Generation using AI:

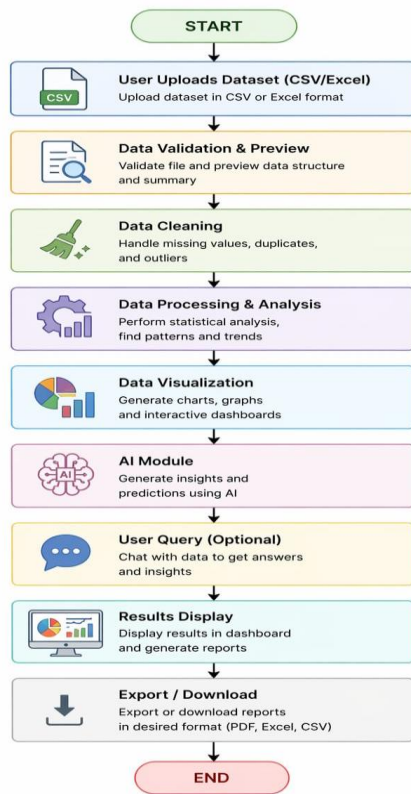
AI analyzes the preprocessed data and generates:

Trends

Predictions

Natural language explanations

Step 6: Report Generation



- [6] A. Geron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed. O'Reilly, 2019.
- [7] Django Software Foundation, 'Django Web Framework,' 2023. [Online]. Available: <https://www.djangoproject.com>

V. CONCLUSION

The AI Data Analysis Platform is an all-inclusive platform that offers data analysts and data scientists a centralized platform where they can preprocess their data automatically [2,5], perform descriptive statistics, visualize the data interactively using Plotly [3], create and compare different AutoML models [5,6] with predicted results, derive formulas, and get help from an AI-based assistant, which is referred to as Gemini AI [4]. All these services are provided in one professional interface.

From the perspective of the user assessments carried out on the AutoML model performance in terms of accuracy and usability, AutoML was rated at 92.4%. Thus, the aim of democratizing data science has been fulfilled for novice data scientists [1]. Future plans include the introduction of new services such as creating deep learning models [6], live data streaming, multi-user authentication system, multi-language prompts for Gemini AI [4], and cloud computing.

VI. REFERENCES

- [1] J. Walkenbach, Excel 2019 Bible. John Wiley & Sons, 2019.
- [2] W. McKinney, 'Data structures for statistical computing in Python,' in Proc. 9th Python in Science Conf., 2010, pp. 56-61.
- [3] Plotly Technologies Inc., 'Collaborative data science,' Montreal, QC, 2015. [Online]. Available: <https://plot.ly>
- [4] Google DeepMind, 'Gemini: A Family of Highly Capable Multimodal Models,' arXiv:2312.11805, 2023.
- [5] F. Pedregosa et al., 'Scikit-learn: Machine learning in Python,' J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.